with a hash function then you can provide a random salt for use in the extract function as an alternative to the nonce, but this is not supported for HKDF-AES-MAC.

In addition to symmetric authenticated encryption, COSE supports a range of public key encryption and signature options, which are mostly very similar to JOSE, so I won't cover them in detail here. One public key algorithm in COSE that is worth highlighting in the context of IoT applications is support ECDH with static keys for both the sender and receiver, known as ECDH-SS. Unlike the ECDH-ES encryption scheme supported by JOSE, ECDH-SS provides sender authentication avoiding the need for a separate signature over the contents of each message. The downside is that ECDH-SS always derives the same key for the same pair of sender and receiver, and so can be vulnerable to replay attacks, reflection attacks, and lacks any kind of forward secrecy. Nevertheless, when used with HKDF and making use of the context fields in table 12.4 to bind derived keys to the context in which they are used, ECDH-SS can be a very useful building block in IoT applications.

## 12.3.2 Alternatives to COSE

Although COSE is in many ways better designed than JOSE and is starting to see wide adoption in standards such as FIDO 2 for hardware security keys (https://fidoalliance.org/fido2/), it still suffers from the same problem of trying to do too much. It supports a wide variety of cryptographic algorithms, with varying security goals and qualities. At the time of writing, I counted 61 algorithm